Grade A+ Health Dashboard: What Each Box Means

A plain-language guide to the 15 boxes on our health dashboard in Sentry: what each one shows, how its number is worked out, what normal looks like, and when someone needs to act. No technical knowledge is needed.

Where to find it: Sentry → **Dashboards** → **Grade A+ health (Epic A)**, in the **Enhanced Electronics** organisation. The other dashboards in that list, such as "Backend Overview" and "Web Vitals", are Sentry's built-in examples, not ours.

Contents

- What the dashboard is for
- Four things to know before you look
- The layout
- Top row: the headline numbers (boxes 1 to 3)
- Is grading working? (boxes 4 to 7)
- Money and records, broken down (boxes 8 and 9)
- Problems people are hitting (boxes 10 to 12)
- Is someone attacking sign-in? (box 13)
- Bugs in the app (boxes 14 and 15)
- When to act: the short list
- The 30-second version
- Good to know

What the dashboard is for

Grade A+ grades students' work with AI, keeps track of teachers' credits, and keeps a record of important actions. This dashboard shows on one screen whether those things are working. It's the first place to look when someone asks "is everything OK?", and it's often where a problem shows up before any user reports it.

Four things to know before you look

1. **The app sends small signals.** Every time something happens, such as a grading finishing or someone typing a wrong password, the app sends Sentry a short note about it. The dashboard adds those notes up.
2. **Rates run from 0 to 1.** 0 means "never happens" and 1 means "always happens". So 0.05 means 5%, or 1 in 20.
3. **An empty box is not broken.** A box stays empty until its event has happened at least once in the time period shown. The period is the last 24 hours by default, and you can change it at the top of the dashboard.

4. **Choose the right environment.** The filter at the top separates the **real site** (`prod`) from the **test site** (`beta`). To see real users only, pick `prod`. See Good to know for the staging exception.

The layout

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ 1 Grading            │ 2 Credit             │ 3 Audit write        │  ← headline: look here first
│   failure rate       │   problems           │   failures           │
├──────────────────────┴──────────┬───────────┴──────────────────────┤
│ 4 Grading failures              │ 5 Gradings                       │
│   over time                     │   (how busy)                     │
├─────────────────────────────────┼──────────────────────────────────┤
│ 6 Backup AI used                │ 7 Unknown AI model               │
├─────────────────────────────────┼──────────────────────────────────┤
│ 8 Credit problems               │ 9 Audit failures                 │
│   by kind                       │   by action                      │
├─────────────────────────────────┴──────────┬───────────────────────┤
│ 10 Error codes over time                   │ 11 Top error codes    │
├────────────────────────────────┬───────────┴──────────────────────┤
│ 12 Server crashes              │ 13 Sign-in attempts held back    │
├────────────────────────────────┼──────────────────────────────────┤
│ 14 Application errors          │ 15 Unresolved bugs               │
└────────────────────────────────┴──────────────────────────────────┘
```

The top row is the headline. The rows below explain it in more detail.

Top row: the headline numbers

Look at these three first. If they look right, things are almost certainly fine.

1. Grading failure rate

What it shows: out of all the times a teacher asked the system to grade work, the share that **failed**, meaning no grade came out.

How it's worked out: each time a grading finishes, the app sends Sentry a **0** if it worked or a **1** if it failed. Sentry takes the **average** of those numbers. The average of a list of 0s and 1s is exactly the share that failed. For example, 19 successes and 1 failure gives 1 ÷ 20 = **0.05**, which is 5%.

The big number covers **the whole time period shown** (24 hours by default). Box 4 shows the same thing hour by hour.

Normal	Act when
Close to 0	The alert fires when, **in the last 15 minutes**, at least **20 gradings** happened and **more than 5%** of them failed

Why 15 minutes and 20 gradings: one failed grading can be bad luck, such as a slow AI reply or a strange file. Looking at a 15-minute window means a single unlucky moment doesn't trigger an alert, but failures that keep happening do. Requiring at least 20 gradings stops a quiet period from causing a false alarm: 1 failure out of 2 gradings is 50%, but it tells us almost nothing.

2. Credit ledger anomalies (credit problems)

What it shows: how many times the system caught **credits being recorded wrongly**. Credits are money, so this is the most urgent box.

How it's worked out: a count. Each time one of the checks below catches a problem, it adds 1.

Normal	Act when
Always exactly 0	**Any number at all.** Act immediately

Box 8 shows which kind of problem it was.

3. Audit write failures

What it shows: the **audit trail** is the system's permanent record of who did what, and when. This box counts how many times saving a record to it **failed**.

How it's worked out: a count. Each failed save adds 1.

Normal	Act when
Always exactly 0	Above 0 for 5 minutes. Things are happening without being recorded. Act immediately

Box 9 shows which action failed to be recorded.

Is grading working?

4. Grading failure rate over time

What it shows: the same measure as box 1, drawn as a line across the day.

What it's for: seeing **when** failures started, and whether they're still happening or have stopped. Read it next to box 5.

5. Gradings

What it shows: how many gradings happened, successful and failed together. It answers "how busy are we?"

Why it matters: it gives box 4 its meaning. Half of 2 gradings failing is probably bad luck. Half of 200 failing is a real problem.

6. Model fallback rate (backup AI used)

What it shows: grading uses a main AI service. When that service is down or overloaded, the work is automatically sent to a **backup AI** instead. This line shows how often the backup had to step in.

How it's worked out: like box 1. Each grading sends a 1 if the backup answered or a 0 if the main AI did, and Sentry averages them. Gradings where we can't tell aren't counted here; they go to box 7.

Normal	Act when
Close to 0	**Above 0.10 (10%)** over 15 minutes. Grades still come out, but our main AI provider is having trouble

7. Unknown model rate

What it shows: how often the system **couldn't tell whether the backup AI was used** for a grading.

Normal	Look into it when
Close to 0	It stays high. Users aren't affected, but our records of which AI graded what are incomplete

Money and records, broken down

When box 2 or box 3 is above zero, these two boxes say what kind of problem it is, so the right person can be called.

8. Credit ledger anomalies by kind

Box 2, split by type of problem. A teacher's credits are kept in **buckets** (this month's allowance, a pack they bought, a top-up), and every movement of credits is written in the **ledger**.

Kind	What happened	In plain terms
`negative_balance`	After a credit movement, the account's balance was below zero	Someone spent credits they didn't have. They got gradings for free, or were charged twice

Kind	What happened	In plain terms
`refund_mismatch`	A refund was due, but the bucket could take back fewer credits than were charged	The teacher may have got back less than they were charged
`refund_no_bucket`	A refund was due, but the bucket the credits came from no longer exists	The teacher was owed credits and got nothing back
`expire_failed`	The scheduled clean-up of unused monthly credits failed for a bucket	Credits that should have run out may still be usable

The box never says **which** teacher was affected. That keeps personal data out of Sentry. A developer finds the teacher from the server log or the credit history for that time.

One limit: these checks are a safety net, not a full audit. For example, the balance is checked once, right after each credit movement. A balance that dips below zero for a split second and then corrects itself won't be seen.

9. Audit write failures by action

Box 3, split by **which action** failed to be recorded, such as a grading or a sign-in. It tells the developers where to look.

Problems people are hitting

10. Error codes over time

What it shows: whenever someone runs into a known problem, the app notes a short **code** for it, such as `WRONG_PASSWORD`, `ACCOUNT_LOCKED` or `FILE_TOO_LARGE`. This chart draws one line per code.

How to read it: many of these are everyday life, because people mistype passwords. What matters is a **sudden jump in one line**. That is often the first sign something broke, before anyone complains.

Normal	Act when
Steady, low lines	One code rises to **more than 3 times** its usual level for an hour

Real example (12 to 15 October): `INVALID_CREDENTIALS` 29, `WRONG_PASSWORD` 13, `ACCOUNT_LOCKED` 6. That is ordinary sign-in noise.

Every code is explained in GRADE_A_PLUS_REASON_CODES.md.

11. Top error codes

The same information as box 10, shown as one bar per code for the whole period. It answers "which problems happened most?"

12. Server errors (SERVER_ERROR)

What it shows: only the `SERVER_ERROR` code, meaning the app **crashed** while someone was using a public page such as sign-in or registration.

Normal	Act when
0, or very rare	**10 or more within 5 minutes.** People may be unable to sign in or register

Is someone attacking sign-in?

13. Sign-in attempts capped (held back)

What it shows: when someone keeps failing to sign in, the system stops writing every failure into the audit trail, so an attacker can't flood it. Instead it **counts the attempts it held back**. There are two lines:

Line	Means
`target`	Many failures against **one account**, for example someone guessing one person's password
`global`	Many failures across **many accounts at once**, which looks like an organised attack

Normal	Act when
0, or a few	**50 or more within 15 minutes**, or **any** `global` at all

Bugs in the app

These two boxes are Sentry's usual job: crashes and bugs, as opposed to user mistakes.

14. Application errors

What it shows: all errors the app reported to Sentry, per hour. A sudden spike means something broke.

15. Top unresolved issues

What it shows: a list of the actual bugs, grouped by Sentry, that nobody has marked as fixed yet, with the most frequent first. Clicking one shows the details. This is the developers' to-do list.

When to act: the short list

Box	Act when	Urgency
2. Credit problems	Any number above 0	**P0**: immediately
3. Audit write failures	Above 0 for 5 minutes	**P0**: immediately
1. Grading failure rate	Above 5% over 15 minutes, with at least 20 gradings	P1: within the hour
6. Backup AI used	Above 10% over 15 minutes	P1: within the hour
10. Error codes	One code more than 3 times its usual level	P1: within the hour
12. Server crashes	10 or more in 5 minutes	P1: within the hour
13. Sign-in attempts held back	50 or more in 15 minutes, or any `global`	P1: within the hour

Once the alerts are switched on, Sentry will email **every member of the Enhanced Electronics organisation** when one of these happens, so nobody has to watch the dashboard all day. The alert rules are in epic_a_alert_rules.md.

The 30-second version

> "Look at the top row. Grading failures should be near zero. Credit problems and audit failures should be exactly zero, and if either isn't, act now. The middle rows show whether grading is healthy, whether our main AI provider is struggling, and what problems users are hitting; watch for a sudden jump in one error code. The bottom rows show whether someone is attacking sign-in, and the list of bugs to fix."

-